

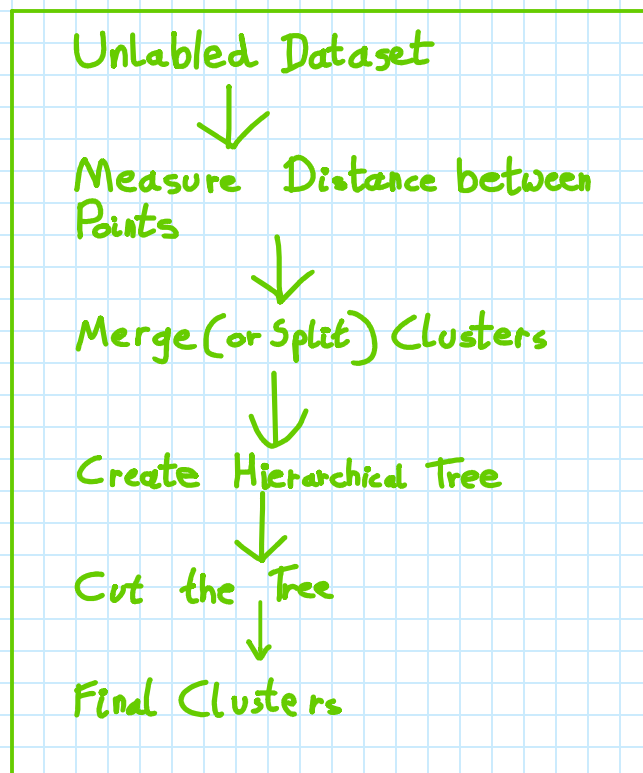
# Hierarchical Clustering

## 1. What is Hierarchical Clustering?

**Hierarchical Clustering** is an **Unsupervised Machine Learning Algorithm** that groups similar data points into clusters by creating a **hierarchical tree-like structure**, called a **Dendrogram**.

Unlike K-Means, Hierarchical Clustering **does not require the number of clusters (K) before training**. Instead, it builds a hierarchy of clusters, and you decide the final number of clusters by cutting the dendrogram.

### Main Idea

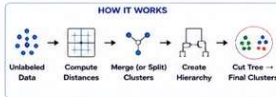


# HIERARCHICAL CLUSTERING

An Unsupervised Learning Algorithm that Builds a Hierarchy of Clusters (Tree)

## 1. WHAT IS HIERARCHICAL CLUSTERING?

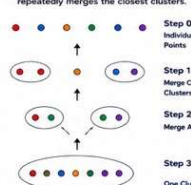
- Hierarchical Clustering creates a hierarchy of clusters by either merging or splitting them.
- It does not require the number of clusters (K) before training.
- The result is a tree-like diagram called a Dendrogram.
- We decide the final number of clusters by cutting the dendrogram at a suitable height.



## 2. TYPES OF HIERARCHICAL CLUSTERING

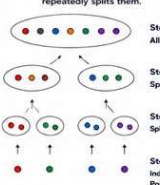
### A. AGGLOMERATIVE (BOTTOM-UP)

Starts with each point as a single cluster and repeatedly merges the closest clusters.

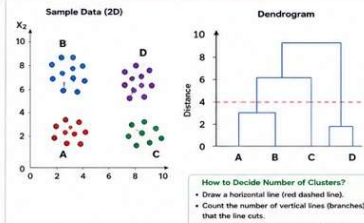


### B. DIVISIVE (TOP-DOWN)

Starts with all points in one cluster and repeatedly splits them.



## 3. DENDROGRAM EXAMPLE



## 4. LINKAGE CRITERIA (How to Measure Distance between Clusters)

**A. SINGLE LINKAGE (Nearest Neighbor)**

Distance between the closest pair of points in the two clusters.

$$D(A,B) = \min_{a \in A, b \in B} d(a,b)$$

Good for: Irregular shapes  
Drawback: Chaining effect

**B. COMPLETE LINKAGE (Farthest Neighbor)**

Distance between the farthest pair of points in the two clusters.

$$D(A,B) = \max_{a \in A, b \in B} d(a,b)$$

Good for: Compact clusters  
Drawback: Sensitive to outliers

**C. AVERAGE LINKAGE (UPGMA)**

Average distance between all pairs of points in the two clusters.

$$D(A,B) = \frac{1}{|A||B|} \sum_{a \in A} \sum_{b \in B} d(a,b)$$

Good for: Balanced clusters

**D. WARD'S LINKAGE (Minimum Variance)**

Merges clusters that result in the minimum increase in within-cluster variance (SS).

Merge that minimizes:  $\Delta ESS = ESS_{new} - ESS_{old}$

$$\Delta ESS = ESS_{new} - ESS_{old}$$

Good for: Compact & spherical clusters (most popular)

## 5. STEP-BY-STEP WORKING (Agglomerative Example)

**Example Data (2D)**

| Point ID | X | Y |
|----------|---|---|
| A        | 1 | 2 |
| B        | 2 | 1 |
| C        | 8 | 7 |
| D        | 7 | 8 |

**Step 0: Each point is a cluster**

**Step 1: Merge Closest (A,B)**

**Step 2: Recompute Distances**

|     | A,B  | C    | D    |
|-----|------|------|------|
| A,B | 0    | 8.19 | 8.73 |
| C   | 8.19 | 0    | 1.41 |
| D   | 8.73 | 1.41 | 0    |

**Step 3: Merge Closest (C,D)**

**Step 4: Merge Final**

Dendrogram represents all these merge steps.

## 6. IMPLEMENTATION (Python - scikit-learn)

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# 1. Data
X = [[1,2], [2,1], [8,7], [7,8]]

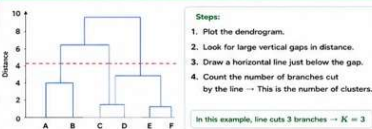
# 2. Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Agglomerative Clustering
model = AgglomerativeClustering(n_clusters=2, # final clusters
                                metric='euclidean',
                                linkage='ward')

labels = model.fit_predict(X_scaled)
print('Cluster Labels:', labels)

# 4. Dendrogram
linked = linkage(X_scaled, method='ward', metric='euclidean')
plt.figure(figsize=(7,4))
dendrogram(linked, labels=[ 'A', 'B', 'C', 'D' ])
plt.title('Dendrogram (Hierarchical Clustering)')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.show()
```

## 7. CHOOSING NUMBER OF CLUSTERS



## 8. SUMMARY

- Builds a hierarchy of clusters (tree structure).
- Does not require K in advance.
- Agglomerative (bottom-up) is most commonly used.
- Uses distance metrics and linkage criteria to merge clusters.
- Dendrogram helps to understand cluster structure and choose K.
- Sensitive to outliers and computationally expensive for large datasets.

## 9. ADVANTAGES

- ✓ No need to specify K before training
- ✓ Dendrogram provides rich information
- ✓ Works well for small datasets
- ✓ Can handle any shape of clusters (with appropriate linkage)

## 10. DISADVANTAGES

- ✗ Computationally expensive ( $O(n^3)$ )
- ✗ Sensitive to noise and outliers
- ✗ Hard to undo merges (once merged)
- ✗ Not suitable for very large datasets